

Local Deal Finder

Project Specification — MVP

Version 1.0 | June 2026

1. Overview

Local Deal Finder is a personal tool that automatically discovers food and retail deals from Reddit, processes them using AI, and surfaces them in a clean feed timed around meal times. The app is built for personal use first, but designed with scaling in mind from the start.

Core value proposition: no existing tool automatically monitors social media at scale, detects deals using AI, and delivers localized results to the user. This fills that gap starting with Reddit as the data source.

2. Scope

MVP Scope (V1)

- Food and restaurant deals only
- Reddit as the sole data source
- GTA/Mississauga subreddits for location targeting
- Single user — no accounts or auth needed
- Scheduled pipeline running 4x daily
- Basic card-based web feed

Out of Scope for MVP

- Instagram, TikTok, Twitter data sources
 - Retail or services categories
 - Push notifications or email summaries
 - Multi-user support
 - Mobile app
 - Map-based interface
-

3. Data Collection Layer

Library: PRAW (Python Reddit API Wrapper)

Target Subreddits

- r/toronto
- r/torontofood
- r/mississauga
- r/GTA
- r/deals (national deals — flagged separately)

Note: Subreddits are configured in config.py and can be added or removed without touching the pipeline logic. Location is inherited implicitly from the subreddit — no per-post location extraction needed for MVP.

What Gets Collected Per Post

- Post title
 - Post body text
 - Source URL (permalink to original Reddit post)
 - Subreddit name
 - Posted timestamp
-

4. Keyword Filter Layer

Purpose: Reduce the number of posts sent to AI to minimize cost. Applied before any AI call.

Method: Simple Python string matching — no ML, no libraries needed. A post passes if it contains at least one keyword (case-insensitive).

Initial Keyword List

- deal, deals
- discount, off (e.g. 50% off)
- free, freebie
- special, promo, promotion
- BOGO, buy one get one
- limited time, today only
- opening, grand opening
- happy hour

Expected reduction: ~10,000 posts down to ~100-300 candidates per run. Keywords are configured in config.py.

5. AI Processing Layer

The AI layer uses a 3-tier cascade to minimize cost while maintaining accuracy.

Tier	Model	Job	Triggers
1	None (free)	Keyword filter	All posts
2	Claude Haiku	Classify: yes / no / uncertain	Posts passing keyword filter
3	Claude Sonnet	Full extraction + uncertain cases	Confirmed deals + uncertain from Haiku

Haiku Classification Prompt (Tier 2)

Haiku receives the post title and body and must return one of three labels:

- yes — clearly a food/restaurant deal or promotion
- no — not a deal
- uncertain — unclear, needs deeper analysis

A deal is defined as: any post promoting a discount, free item, limited time offer, or grand opening tied to a real food business.

Sonnet Extraction Prompt (Tier 3)

Sonnet receives confirmed and uncertain posts and returns structured JSON with the following fields:

- is_deal (boolean) — final confirmation
- business_name (string) — name of the restaurant or food business
- deal_description (string) — plain English description of the deal
- location (string) — city or neighbourhood if mentioned, else null
- urgency (enum) — limited_time / ongoing / unknown
- source_url (string) — passed through from collection
- subreddit (string) — passed through from collection

Cost Optimization Notes

- Batch multiple posts per API call where possible
- Cache results — never re-process a post already in the database

- Post-MVP: evaluate if Haiku can handle simple extractions to reduce Sonnet usage

6. Database

Database: SQLite — zero setup, file-based, easy to migrate to PostgreSQL later if scaling.

Deals Table Schema

Field	Type	Description
id	INTEGER PRIMARY KEY	Auto-incremented unique ID
business_name	TEXT	Name of the restaurant or food business
deal_description	TEXT	Plain English deal description
location	TEXT / NULL	City or neighbourhood, null if unknown
source_url	TEXT	Link to original Reddit post
subreddit	TEXT	Subreddit it was found in
posted_at	DATETIME	When the original post was made on Reddit
fetches_at	DATETIME	When our app found and processed it
urgency	TEXT	limited_time / ongoing / unknown
is_expired	BOOLEAN	True if deal is no longer active, default false

Expiry Logic

- If urgency = limited_time and fetched_at is older than 48 hours → mark is_expired = true
- Expired deals are hidden from the feed by default
- User can manually dismiss a deal (sets is_expired = true)

7. Scheduler

Library: APScheduler (Python). Runs the full pipeline automatically at meal-time intervals.

Run	Time	Rationale
Morning	6:00 AM	Breakfast and morning coffee deals
Lunch	11:00 AM	Lunch planning window

Dinner	4:00 PM	Dinner planning window
Late Night	9:00 PM	Late night and next-day planning

Each run executes the full pipeline: collect → filter → classify → extract → store. Previously processed posts are skipped via URL deduplication check against the database.

8. Backend API

Framework: FastAPI (Python). Lightweight REST API that serves deal data to the frontend.

Endpoints (MVP)

Method	Route	Description
GET	/deals	Returns all active (non-expired) deals, newest first
GET	/deals?location=Etobicoke	Filter deals by location
GET	/deals?urgency=limited_time	Filter by urgency
POST	/deals/{id}/dismiss	Mark a deal as expired/dismissed

9. Frontend

Framework: React + Tailwind CSS. Simple, clean card-based feed. No complex UI for MVP — function over form.

Deal Card — Fields Displayed

- Business name
- Deal description
- Location (if available)
- Urgency tag (e.g. Limited Time)
- Time since posted (e.g. 2 hours ago)
- Source link — opens original Reddit post
- Dismiss button — removes card from feed

Feed Features

- Default sort: newest first
- Expired/dismissed deals hidden by default

- Subreddit tag visible on each card (so user knows origin)

10. Tech Stack Summary

Layer	Technology	Cost
Data Collection	Python + PRAW	Free
Keyword Filter	Python (built-in)	Free
AI Classification	Claude Haiku (Anthropic API)	~\$0.001 / 1k tokens
AI Extraction	Claude Sonnet (Anthropic API)	~\$0.003 / 1k tokens
Scheduler	APScheduler	Free
Database	SQLite	Free
Backend	FastAPI	Free
Frontend	React + Tailwind	Free
Estimated monthly cost	Anthropic API only	< \$2 / month at MVP scale

11. Project Folder Structure

File / Folder	Purpose
collector/reddit.py	Connects to Reddit via PRAW and pulls posts from target subreddits
filter/keyword.py	Filters posts by keyword before any AI call
ai/classifier.py	Sends filtered posts to Haiku for yes/no/uncertain classification
ai/extractor.py	Sends confirmed deals to Sonnet for structured field extraction
database/models.py	Defines the deals table schema
database/db.py	Handles saving and reading deals from SQLite
scheduler/jobs.py	Runs the full pipeline at 6am, 11am, 4pm, 9pm
api/routes.py	FastAPI endpoints serving deal data to the frontend
frontend/src/	React + Tailwind web UI
config.py	All settings: subreddits, keywords, API keys, schedule times
main.py	Entry point — launches the app

12. Recommended Build Order

Build and test each layer independently before connecting them. Each step should work in isolation.

Step	What to Build	Done When...
1	Reddit collector (reddit.py)	Can pull and print posts from target subreddits
2	Keyword filter (keyword.py)	Correctly filters a sample of posts down to candidates
3	Haiku classifier (classifier.py)	Returns yes/no/uncertain labels on sample posts
4	Sonnet extractor (extractor.py)	Returns valid JSON deal objects from confirmed posts
5	Database (models.py + db.py)	Can save and retrieve deals from SQLite
6	Scheduler (jobs.py)	Full pipeline runs on schedule without errors
7	FastAPI backend (routes.py)	Can hit /deals and get JSON back in browser
8	React frontend	Cards render correctly with real data from API

13. Scaling Notes (Post-MVP)

The MVP is built for one user but the following design decisions make scaling easier later:

- User location and preferences stored in config/DB, not hardcoded
- Subreddits and keywords are configurable — easy to add more sources
- SQLite can be swapped for PostgreSQL with minimal code changes
- API key billing: at scale, explore per-user keys or a subscription model to cover Anthropic costs
- Post-MVP categories: retail, services, events can be added as a new field + UI filter
- Post-MVP sources: Instagram, TikTok, Twitter can plug into the same pipeline with a new collector module
- AI optimization: re-evaluate Haiku vs Sonnet split based on real accuracy data after MVP